



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorio de Computacion Salas A y B

Profesor(a):

JOSE ANTONIO AYALA BARBOSA

Asignatura:

Programación orientada a objetos



Grupo:

02

No de practica(s):

Practica 12 : Hilos

Integrante(s):

Santos Carmona Valeria Sofia | Hernandez Reyes Rebeca Sarai | Al

No de lista o brigada:

Semestre:

2026-1

Fecha de entrega:

20 DE NOVIEMBRE DEL 2025

Observaciones:

Calificacion:

Práctica 12 Hilos

Integrantes : Santos Carmona Valeria Sofia | Hernandez Reyes Rebeca Sarai | Almaguer Miranda Michel Santiago

1

I.Objetivo

Implementar el concepto de multitarea utilizando hilos en un lenguaje orientado a objetos.

CODIGO Y DESARROLLO

```
package poop12;

/**
 *
 * @author Michel
 */
public class POOP12 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Hilo hilo1 = new Hilo("primer hilo");
        hilo1.start();

        new Hilo("segundo hilo").start();
        System.out.println("Termina hilo principal");
    }
}
```

Clase Principal

```
public static void main(String[] args) {
    Hilo hilo1 = new Hilo("primer hilo");
    hilo1.start();
}
```

Para la ejecución de los hilos en la clase Main se inicializa la ejecución multitarea y se llama el método start() que es el que llama a su vez a run() de manera interna aunque lo hace en un nuevo hilo de ejecución que corre paralelo al hilo principal que es nuestro main.

El código ejecuta dos hilos (primer y segundo hilo) para que se ejecuten con el ciclo de 10 iteraciones que definimos previamente en run() de forma simultánea al hilo principal.

Los hilos secundarios se ejecutan en el for loop al mismo tiempo y su salida no se verá primero las 10 iteraciones de un hilo y luego las 10 del otro si no que las imprimirá entrelazadas de forma determinada lo que demuestra la multitarea.

```
package poop12;

/**
 *
 * @author Michel
 */
public class Hilo extends Thread {
    public Hilo (String name){
        super(name);
    }
    @Override
    public void run(){
        for (int i = 0; i < 10; i++) {
            System.out.println("Iteracion "+i+" de "+ getName());
        }
        System.out.println("Termina el "+getName());
    }
}
```

Clase Hilo

Clase Hilo extiende de la clase Thread

```
@Override
public void run(){
```

el código que se encuentra dentro del run() hace que el hilo ejecute un ciclo for que hace una iteración de 10 veces donde imprime el número de iteración y el nombre del hilo de manera respectiva además de tener un constructor llamado super(name) que asigna el nombre al hilo para facilitarle al usuario su visualización.

```

package poop12;

public class POOP12 {

    public static void main(String[] args) {
        Contador contadorCompartido = new Contador();

        Hilo hilo1 = new Hilo("primer hilo", contadorCompartido);
        Hilo hilo2 = new Hilo("segundo hilo", contadorCompartido);

        hilo1.start();
        hilo2.start();

        try {
            hilo1.join();
            hilo2.join();
        } catch (InterruptedException e) {
            System.out.println("El hilo principal fue interrumpido.");
        }

        System.out.println("Termina hilo principal");
        System.out.println("Valor FINAL del Contador Compartido: " + contadorCompartido.getValor());
    }
}

```

Clase Main: El cambio principal de esta clase es la creación única donde se crea una sola instancia de Contador su propósito es pasar la única instancia a los dos hilos (hilo1,hilo2) y se asegura que los dos operen en el mismo estado además de utilizar el método join () para esperar que los dos terminen las 10 iteraciones y se imprime el resultado final así que si la sincronización es correcta el valor que debe lanzar al final es de 20 que es el caso en la imagen del ejecutable.

```

package poop12;

public class Hilo extends Thread {
    private Contador contador;

    public Hilo (String name, Contador contador) {
        super(name);
        this.contador = contador;
    }

    @Override
    public void run() {
        System.out.println("Inicia el " + getName());

        for (int i = 0; i < 10; i++) {
            contador.incrementar();
            System.out.println("Iteracion " + (i+1) + " de " + getName() +
                               " | Contador actual: " + contador.getValor());
        }

        System.out.println("Termina el " + getName() +
                           " | Valor final esperado: " + contador.getValor());
    }
}

```

Clase Hilo: El cambio principal que sufrió esta clase fue la inyección de dependencia donde ahora recibe una referencia al objeto contador compartido en su constructor su propósito es permitir que la tarea del hilo run() acceda al mismo objeto que es el contador y llama repetidamente al método sincronizado contador.incrementar()) dentro de su bucle run()que fuerza a los hilos a “pelear” por así decirlo por el recurso.

Ejecutable

```

Termina hilo principal
Iteracion 0 de segundo hilo
Iteracion 0 de primer hilo
Iteracion 1 de primer hilo
Iteracion 2 de primer hilo
Iteracion 3 de primer hilo
Iteracion 1 de segundo hilo
Iteracion 4 de primer hilo
Iteracion 5 de primer hilo
Iteracion 6 de primer hilo
Iteracion 7 de primer hilo
Iteracion 8 de primer hilo
Iteracion 9 de primer hilo
Iteracion 2 de segundo hilo
Iteracion 3 de segundo hilo
Iteracion 4 de segundo hilo
Iteracion 5 de segundo hilo
Iteracion 6 de segundo hilo
Iteracion 7 de segundo hilo
Iteracion 8 de segundo hilo
Iteracion 9 de segundo hilo
Termina el primer hilo
Termina el segundo hilo

```

ACTIVIDAD EXTRA: *Implementar métodos sincronizables.

```
package poop12;

public class Contador {
    private int valor;

    public Contador() {
        this.valor = 0;
    }

    public synchronized void incrementar() {
        this.valor++;
        try {
            Thread.sleep(10);
        } catch (InterruptedException e) {
            System.out.println("Error en el sleep");
        }
    }

    public int getValor() {
        return valor;
    }
}
```

Clase agregada Contador:

El propósito de por qué creamos esta clase es para asegurarnos que todos los hilos modifiquen el mismo dato sin esto la multitarea no tendría un desafío para demostrar la sincronización.

Contiene un método sincronizado

```
public synchronized void incrementar() {
```

y tiene un recurso compartido que es un objeto independiente que contiene el dato que los hilos múltiples manipulan que es el valor

EJECUTABLE

```
Inicia el segundo hilo
Inicia el primer hilo
Iteracion 1 de segundo hilo | Contador actual: 1
Iteracion 1 de primer hilo | Contador actual: 2
Iteracion 2 de segundo hilo | Contador actual: 3
Iteracion 2 de primer hilo | Contador actual: 4
Iteracion 3 de segundo hilo | Contador actual: 5
Iteracion 3 de primer hilo | Contador actual: 6
Iteracion 4 de segundo hilo | Contador actual: 7
Iteracion 4 de primer hilo | Contador actual: 8
Iteracion 5 de segundo hilo | Contador actual: 9
Iteracion 5 de primer hilo | Contador actual: 10
Iteracion 6 de segundo hilo | Contador actual: 11
Iteracion 6 de primer hilo | Contador actual: 12
Iteracion 7 de segundo hilo | Contador actual: 13
Iteracion 7 de primer hilo | Contador actual: 14
Iteracion 8 de segundo hilo | Contador actual: 15
Iteracion 8 de primer hilo | Contador actual: 16
Iteracion 9 de segundo hilo | Contador actual: 17
Iteracion 9 de primer hilo | Contador actual: 18
Iteracion 10 de segundo hilo | Contador actual: 19
Termina el segundo hilo | Valor final esperado: 20
Iteracion 10 de primer hilo | Contador actual: 20
Termina el primer hilo | Valor final esperado: 20
Termina hilo principal
Valor FINAL del Contador Compartido: 20
```

El propósito de la sincronización fue evitar las condiciones de carrera que es cuando dos o mas hilos intentan modificar el mismo valor de manera simultanea lo que podría provocar que se pierdan.

Conclusiones:

En conclusión la practica cumplió con los objetivos de implementar la multitarea utilizando hilos en la primera parte de la practica ocupamos la multitarea al crear la clase hilo que extiende de Thread donde llamamos al método start() en las instancias de primer y segundo hilo y se lanza en la ejecución del método run() de forma concurrente a la par del hilo principal y en la actividad extra usamos la sincronización donde gestionamos recursos compartidos y evitamos que se hicieran condiciones de carrera y al declarar el método incrementar() como lo es synchronized garantizamos que solo un hilo a la vez pueda modificar el estado interno del contador.

Referencias:

Oracle. (s. f.). *Concurrency: Threads*. The Java Tutorials. Recuperado de <https://docs.oracle.com/javase/tutorial/essential/concurrency>

Tutoriales Programación Ya. (s. f.). *Hilos en Java - método synchronized*. Recuperado de <https://m.youtube.com/watch?v=qfCn9JR69bc>.

Oracle. (s. f.). *Concurrency: Synchronization*. The Java Tutorials. Recuperado de <https://docs.oracle.com/javase/tutorial/essential/concurrency/sync.html>